

Speculative Decoding

A small model guesses, a big model checks · video 2:27

This lesson answers one question: how can a big model write faster without changing its output? A small **draft** model guesses a few tokens ahead; the big **target** model checks them all in one pass and keeps the ones it agrees with. On Python code, Qwen2.5-1.5B wrote the same 128 tokens 2.3 times faster; on prose, guessing is harder and too many guesses made it slower. With sampling, an accept/reject rule keeps the target's distribution exactly.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 6).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Why checking is cheap	▶ 0:08-0:41	1	☐
2. The algorithm and its speed	▶ 0:41-1:48	3	☐
3. Sampling: keeping the distribution exact	▶ 1:48-2:18	2	☐

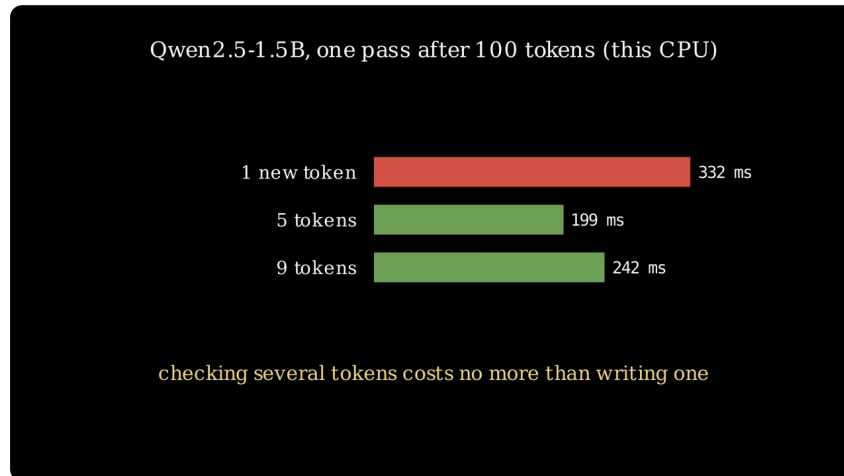
Before you start: Deep Dive episodes 31 (the KV cache, prefill vs decode) and 32 (batching). Code: `code/d34_speculative` (Qwen2.5-0.5B and 1.5B, about 8 GB in RAM).

CONCEPT 1 OF 3

Why checking is cheap

▶ Video 0:08–0:41

A big model writes one token per pass, and each decode pass is slow because it reads all the weights for little math (episode 31). But a pass over several new tokens reads the same weights once. On this CPU, one Qwen2.5-1.5B pass for 1 new token took **332 ms**; for 5 tokens, **199 ms**; for 9, **242 ms**. (One token is even slower here, the same matrix-vector oddity as episode 32; on a GPU it is about equal.)



▶ 0:40 One Qwen2.5-1.5B pass after 100 tokens: 332 ms for 1 new token, 199 ms for 5, 242 ms for 9 (this CPU).

KEY IDEA

Checking a handful of tokens costs about as much as writing one.

Check 1: Why checking is cheap

Answer all 1 questions before looking at the answers. Pass mark: all correct.

1.1 TRUE OR FALSE

“Speculative decoding (greedy) can change the big model's output.”

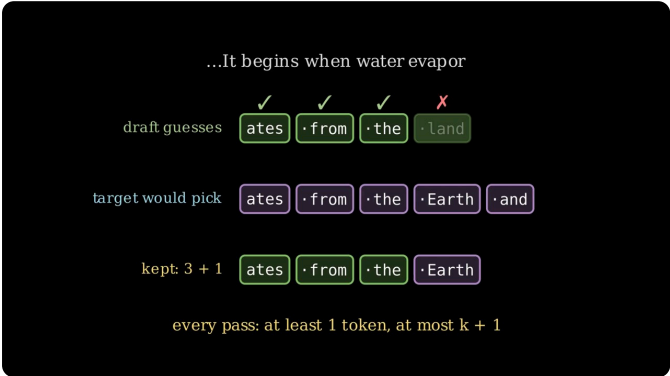
True ☐ False ☐

CONCEPT 2 OF 3

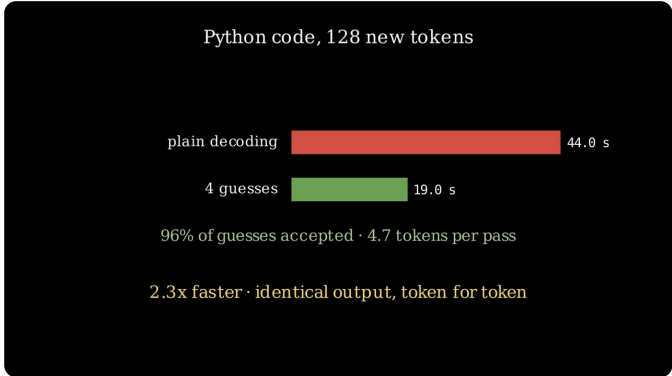
The algorithm and its speed

▶ Video 0:41-1:48

Each round, the draft (Qwen2.5-0.5B) guesses k tokens one by one. The target reads all of them in a single pass and, at every position, gives the token it would have picked. Keep the guesses up to the first disagreement, then add the target's own token: every pass yields at least 1 token and at most k + 1.

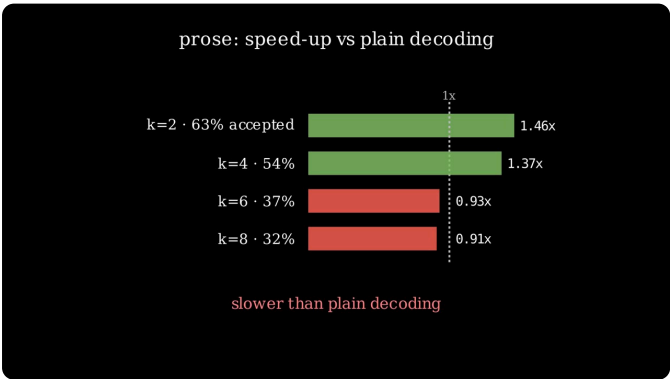


▶ 0:59 A real round: the draft guesses “ates · from · the · land”; the target agrees on 3, then adds its own “Earth”.

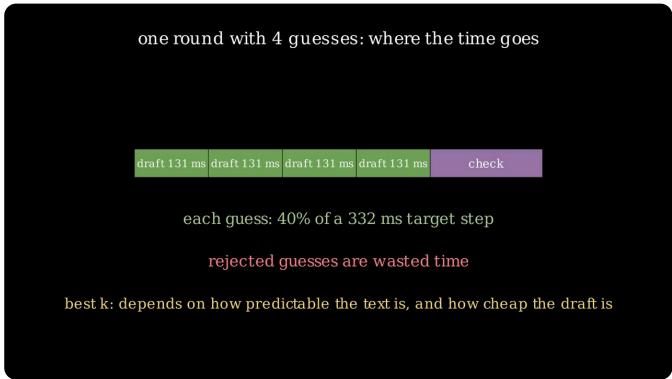


▶ 1:17 Python code: 44.0 s plain, 19.0 s with 4 guesses (96% accepted, 4.7 tokens per pass).

128 TOKENS	ACCEPTED	TOKENS PER PASS	TIME	SPEED-UP
code, plain	-	1	44.0 s	1x
code, k = 4	96%	4.7	19.0 s	2.31x
prose, plain	-	1	42.8 s	1x
prose, k = 2	63%	2.2	29.3 s	1.46x
prose, k = 4	54%	3.1	31.3 s	1.37x
prose, k = 6	37%	3.2	45.9 s	0.93x
prose, k = 8	32%	3.6	47.3 s	0.91x



▶ 1:31 Prose: 1.46x with 2 guesses (63% accepted), slower than plain with 6 or 8.



▶ 1:47 Each draft token costs 131 ms, 40% of a 332 ms target step.

The guesses aren't free: the draft takes 131 ms per token here, 40% of a target step, and every rejected guess is wasted time. The best k depends on how predictable the text is and how cheap the draft is.

KEY IDEA
Speed-up $\approx$ tokens per pass $\div$ (cost of k draft steps + one check, in target steps).

Check 2: The algorithm and its speed
Answer all 3 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION
k = 4, and the first 2 guesses match the target. How many tokens does this pass add?
.....

2.2 CALCULATION
Estimate the code speed-up with k = 4: a round costs 4 draft steps (131 ms) and one 5-token check (199 ms) and yields 4.7 tokens; plain decoding takes 332 ms per token.
.....

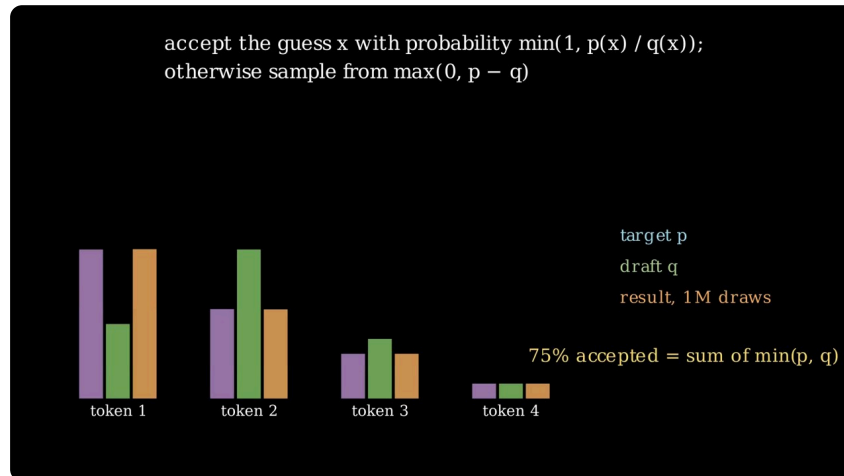
2.3 SHORT ANSWER
Why did 6 guesses make prose slower than plain decoding?
.....

CONCEPT 3 OF 3

Sampling: keeping the distribution exact

▶ Video 1:48-2:18

With sampling, the draft samples x from its distribution q ; accept it with probability $\min(1, p(x) / q(x))$, where p is the target's distribution. If it is rejected, sample from what's left over, $\max(0, p - q)$ renormalised. The result follows p exactly. A million simulated draws with $p = (0.50, 0.30, 0.15, 0.05)$ and $q = (0.25, 0.50, 0.20, 0.05)$ gave $(0.501, 0.299, 0.150, 0.050)$, and 75.0% of guesses were accepted: the sum of $\min(p, q)$.



▶ 2:08 Accept with probability $\min(1, p/q)$, else sample from $\max(0, p - q)$: 1M draws match the target; 75% accepted.

In code, the greedy check is a short loop: run the target once over the guesses, count how many match its own choices, and keep those plus its next token.

KEY IDEA

The acceptance rate is the overlap between the draft's and the target's distributions.

Check 3: Sampling: keeping the distribution exact

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

With $p = (0.50, 0.30, 0.15, 0.05)$ and $q = (0.25, 0.50, 0.20, 0.05)$: what fraction of guesses is accepted, and which token is drawn after a rejection?

3.2 HANDS-ON CODE

Try it yourself. Check that the rule reproduces p .

```
import torch
p = torch.tensor([0.50, 0.30, 0.15, 0.05]); q = torch.tensor([0.25, 0.50, 0.20, 0.05])
x = torch.multinomial(q, 100_000, replacement=True)
keep = torch.rand(100_000) < (p[x] / q[x]).clamp(max=1)
r = (p - q).clamp(min=0)
y = torch.where(keep, x, torch.multinomial(r / r.sum(), 100_000, replacement=True))
print(torch.bincount(y) / 100_000)
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Why checking is cheap · rewatch ▶ 0:08-0:41

1.1 False. Only tokens the target itself would have picked are kept; in all 8 runs the output was identical token for token.

2. The algorithm and its speed · rewatch ▶ 0:41-1:48

2.1 3. The 2 accepted guesses plus the target's own token at the first disagreement.

2.2 About 2.2x. $4 \times 131 + 199 = 723$ ms for 4.7 tokens = 154 ms per token; $332 / 154 \approx 2.2$ (measured 2.31x).

2.3 Only 37% were accepted, so most of the 6 draft steps (131 ms each) were wasted, while each pass still gained only about 3 tokens. Draft cost grows with k ; accepted tokens stop growing.

3. Sampling: keeping the distribution exact · rewatch ▶ 1:48-2:18

3.1 75%; always token 1. $\sum \min(p, q) = 0.25 + 0.30 + 0.15 + 0.05 = 0.75$; $\max(0, p - q) = (0.25, 0, 0, 0)$.

3.2 About [0.50, 0.30, 0.15, 0.05]. Accepted mass $\min(p, q)$ plus the leftover $p - q$ on rejection adds up to p .